SOFTWARE TESTING • STLab02

EDUCOMMERCE

TEST CASE EXECUTION

Requirement → Test Condition → Test Scenario → Test Case → Test Execution

---

BÁO CÁO BÀI THỰC HÀNH STLab02

Python • CSV Data-driven Testing • Test Reporting

MSSV	.....
Họ và tên	.....
Lớp	.....
Môn học	Software Testing
Phạm vi REQ	REQ-01, 02, 03, 04, 09, 10, 14, 16
GitHub URL	CHƯA PUBLISH - cập nhật URL sau khi tạo repo public

Báo cáo dùng module EduCommerce trong Instruction và giữ nguyên logic nghiệp vụ mẫu để quan sát kết quả. Bộ test được chạy local bằng Python; chưa có phiên Colab hoặc tài khoản GitHub được kết nối trong phiên làm việc này. Vì vậy không giả lập ảnh chụp Colab/GitHub và không tự tạo URL repository không có thật.

Phong cách trình bày theo practice guide STLab02 Instruction: tiêu đề rõ, bảng có header, ID traceability thống nhất và kết quả thực thi tách biệt expected gốc.

MỤC LỤC

- 1. Thông tin sinh viên, phạm vi và tóm tắt kết quả
- 2. Application overview và Selected Requirements
- 3. Requirement Analysis & Acceptance Rules
- 4. Test Condition, Test Scenario Matrix và Traceability
- 5. Detailed Test Cases, Test Data và Expected
- 6. Python Test Architecture
- 7. Execution Evidence và Test Run Information
- 8. Test Report: Summary, Chart và Coverage
- 9. Root Cause / Defect Candidate / Retest
- 10. Problem Solving
- 11. Interview Questions
- 12. Self-review và GitHub Repository

1. THÔNG TIN SINH VIÊN, PHẠM VI & KẾT QUẢ

Thông tin	Giá trị
MSSV / Họ tên / Lớp	Điền trước khi nộp
Ứng dụng	EduCommerce - mô phỏng checkout, không có website/database thật
Selected REQ	REQ-01, REQ-02, REQ-03, REQ-04, REQ-09, REQ-10, REQ-14, REQ-16
Không chọn lại	REQ-06, REQ-08, REQ-11 (đã là worked examples trong Instruction)
Test data	test_cases.csv - 24 dòng, fixture mới cho từng case
Execution environment	Python 3.13.5; thực thi local; timestamp 2026-11-02T21:39:28+07:00
Result	24 PASS / 0 FAIL / 0 BLOCKED; pass rate 100.0%
Test report	reports/test_execution.csv; reports/summary_by_requirement.csv; reports/test_dashboard.html
GitHub	Chưa publish; repository name đề xuất: stlab02-educommerce-testing

Pass rate 100% chỉ áp dụng cho 24 expected assertions thuộc 8 REQ đã chọn. Snapshot state vẫn ghi nhận stock giảm trong ca payment declined và payment method không hợp lệ; đây là quan sát liên module cần được nêu riêng, không bị che bởi PASS.

2. APPLICATION OVERVIEW & SELECTED REQUIREMENTS

EduCommerce mô phỏng hành trình mua hàng: Catalog → Cart → Promotions/Shipping → Inventory → Payments → Checkout → Order. Theo Instruction, ứng dụng chạy trong bộ nhớ, không yêu cầu xây website hay database. Các testcase tạo dữ liệu riêng bằng fixture().

Module / class	Input / function chính	Rule / state được quan sát
Catalog	get(sku)	Chỉ Product active mới được truy vấn để mua.
Cart	add(sku, quantity); subtotal()	Quantity nguyên dương; cùng SKU cộng dồn; subtotal dùng Decimal.
Shipping	calculate(subtotal, area)	Inner >= 500.000 miễn ship; inner dưới ngưỡng 30.000; outer 50.000.
Payments	collect(amount, method, approved, key)	COD pending; ONLINE chỉ paid khi approved; method khác bị reject.
Checkout	place(customer, cart, ...)	Kiểm tra login/cart, tính tiền, reserve stock, payment, tạo Order.
Order	order_id, customer_id, lines, status, total	Ghi trạng thái và chi phí cho đơn phát sinh.

2.1. Dữ liệu sản phẩm mẫu

SKU	Tên sản phẩm	Đơn giá	Stock ban đầu	Trạng thái
A	Wireless Mouse	200.000 VND	20	Active
B	Mechanical Keyboard	300.000 VND	15	Active
C	USB-C Hub	150.000 VND	12	Active

2.2. Phạm vi và giới hạn

• Phạm vi chính: Catalog, Cart, Pricing, Authentication, Shipping, Payment, Cart Integrity và Payment Safety.

• Không chọn REQ-06, REQ-08, REQ-11 làm REQ chấm chính vì Instruction đã làm mẫu ba nội dung này.

• Ứng dụng in-memory không mô phỏng concurrency, network failure thật, database transaction, gateway production hay security/performance chuyên sâu.

• Các expected được xây từ acceptance criteria, không lấy actual của code làm expected.

3. REQUIREMENT ANALYSIS & ACCEPTANCE RULES

Bảng dưới đây phân tích actor, điều kiện trước, input/output, risk và open question cho 8 REQ được chọn.

REQ / Module	Actor	Acceptance rule	Precondition / Input	Expected output	Risk	Question / ambiguity
REQ-01 Catalog	User / Catalog	Chỉ sản phẩm active mới có thể truy vấn để mua.	Catalog mặc định chứa SKU; thao tác truy vấn theo SKU.	Product trả về khi active; PRODUCT_UNAVAILABLE nếu inactive/không có.	Sản phẩm ngừng kinh doanh vẫn được đưa vào đơn.	UI message khi SKU inactive chưa mô tả.
REQ-02 Cart	User / Cart	Quantity phải là số nguyên dương; cùng SKU được cộng dồn.	Cart được khởi tạo; SKU hợp lệ; quantity nhận từ thao tác add.	Một CartLine/SKU với tổng quantity; dữ liệu không hợp lệ bị từ chối.	Sai số lượng hoặc cùng SKU tạo nhiều line, subtotal sai.	Giới hạn tối đa cho một line/giỏ hàng chưa nêu.
REQ-03 Pricing	System / Cart	Subtotal = tổng price × quantity và dùng Decimal.	Một hoặc nhiều CartLine; Product.price là Decimal.	Subtotal kiểu Decimal và giá trị khớp phép nhân/cộng.	Sai số tiền nếu dùng float hoặc chuyển kiểu không đúng.	Quy tắc rounding cho thuế/phí ngoài subtotal chưa mô tả.
REQ-04 Authentication	Customer / Checkout	Chỉ khách đã đăng nhập được checkout.	Cart có item; authenticated true/false.	Guest nhận LOGIN_REQUIRED; user đã đăng nhập được đi tiếp.	Tạo order không gắn phiên/tài khoản hợp lệ.	Requirement không đề cập guest cart; ngoài scope.
REQ-09 Shipping	System / Shipping	Inner: subtotal >= 500.000 miễn ship; dưới ngưỡng 30.000; outer 50.000.	Subtotal Decimal; area inner/outer.	Phí ship trả về theo khu vực và ngưỡng.	Tính phí sai làm total sai.	Instruction quy định dùng subtotal trước voucher/discount.
REQ-10 Payment	Customer / Payments / Checkout	COD = PENDING_COD; ONLINE chỉ PAID khi approved; declined không tạo order.	User đăng nhập; cart hợp lệ; method COD/ONLINE; approved true/false.	COD pending; online approved paid; declined reject và không tạo order.	Payment/order status sai; order tạo dù payment declined.	Timeout/unknown response của gateway không mô phỏng trong lab.
REQ-14 Cart Integrity	Customer / Checkout	Cart rỗng không được checkout.	Cart.lines rỗng; gọi Checkout.place().	EMPTY_CART và không tạo order.	Order không có mặt hàng.	Thông điệp UI khi cart rỗng chưa nêu.
REQ-16 Payment Safety	Customer / Payments	Method ngoài COD/ONLINE phải bị từ chối.	User đã đăng nhập; cart hợp lệ; gửi method khác.	INVALID_PAYMENT_METHOD; không tạo order.	Phương thức không hỗ trợ lọt vào luồng tạo đơn.	Danh sách phương thức có thể mở rộng trong tương lai chưa nêu.

Những câu hỏi được giữ ở dạng open question vì tài liệu không mô tả: giới hạn quantity tối đa, UI message khi sản phẩm inactive, quy tắc guest cart, xử lý timeout gateway, hoặc danh sách payment method mở rộng. Không tự đặt expected result cho các điểm này.

4. TEST CONDITION, TEST SCENARIO MATRIX & TRACEABILITY

Test Condition xác định khía cạnh cần kiểm tra; Test Scenario mô tả tình huống nghiệp vụ; Test Case cung cấp input, precondition, expected cụ thể. Mỗi case có ID liên kết xuyên suốt REQ → COND → TS → TC → RUN.

4.1. Test Condition

Requirement	Condition ID	Condition
REQ-01	COND-CAT-01	Trạng thái active của SKU
REQ-01	COND-CAT-02	SKU inactive hoặc không tồn tại
REQ-02	COND-CART-01	Quantity nguyên dương
REQ-02	COND-CART-02	Cộng dồn / quantity không hợp lệ
REQ-03	COND-PRICE-01	Phép tính Decimal nhiều SKU
REQ-03	COND-PRICE-02	Thay đổi giá/độ chính xác thập phân
REQ-04	COND-AUTH-01	Customer chưa đăng nhập
REQ-04	COND-AUTH-02	Customer đã đăng nhập
REQ-09	COND-SHP-01	Inner zone và ngưỡng 500.000
REQ-09	COND-SHP-02	Outer zone phí cố định
REQ-10	COND-PAY-01	COD state
REQ-10	COND-PAY-02	Online approved/declined
REQ-14	COND-EMPTY-01	Cart rỗng
REQ-14	COND-EMPTY-02	Cart có item
REQ-16	COND-METHOD-01	Method ngoài danh sách
REQ-16	COND-METHOD-02	Method được hỗ trợ

4.2. Test Scenario Matrix

Scenario ID	REQ	Condition	What to test	Kind	Priority	TC ID
TS-CAT-01	REQ-01	COND-CAT-01	Tra cứu sản phẩm đang active	Positive	P1	TC-CAT-001
TS-CAT-02	REQ-01	COND-CAT-02	Từ chối sản phẩm inactive	Negative	P1	TC-CAT-002
TS-CAT-03	REQ-01	COND-CAT-02	Từ chối SKU không tồn tại	Negative	P1	TC-CAT-003
TS-CART-01	REQ-02	COND-CART-01	Thêm số lượng nguyên dương vào giỏ	Positive	P1	TC-CART-001
TS-CART-02	REQ-02	COND-CART-02	Cộng dồn cùng SKU trong cart	Positive	P1	TC-CART-002
TS-CART-03	REQ-02	COND-CART-02	Từ chối quantity bằng 0	Negative	P1	TC-CART-003
TS-PRICE-01	REQ-03	COND-PRICE-01	Tính subtotal nhiều SKU bằng Decimal	Positive	P1	TC-PRICE-001
TS-PRICE-02	REQ-03	COND-PRICE-01	Giữ chính xác phần thập phân khi nhân giá	Positive	P1	TC-PRICE-002
TS-PRICE-03	REQ-03	COND-PRICE-02	Subtotal phản ánh đúng giá sản phẩm cập nhật	Positive	P1	TC-PRICE-003
TS-AUTH-01	REQ-04	COND-AUTH-01	Guest không được checkout bằng COD	Negative	P0	TC-AUTH-001
TS-AUTH-02	REQ-04	COND-AUTH-01	Guest không được checkout bằng ONLINE	Negative	P0	TC-AUTH-002
TS-AUTH-03	REQ-04	COND-AUTH-02	Customer đăng nhập được tiếp tục checkout	Positive	P1	TC-AUTH-003
TS-SHP-01	REQ-09	COND-SHP-01	Inner area dưới ngưỡng miễn ship	Boundary	P1	TC-SHP-001
TS-SHP-02	REQ-09	COND-SHP-01	Inner area tại đúng ngưỡng miễn ship	Boundary	P0	TC-SHP-002
TS-SHP-03	REQ-09	COND-SHP-02	Outer area áp dụng phí ship cố định	Positive	P1	TC-SHP-003
TS-PAY-01	REQ-10	COND-PAY-01	COD tạo order trạng thái PENDING_COD	Positive	P0	TC-PAY-001
TS-PAY-02	REQ-10	COND-PAY-02	Online approved tạo trạng thái PAID	Positive	P0	TC-PAY-002
TS-PAY-03	REQ-10	COND-PAY-02	Online declined không tạo order	Negative	P0	TC-PAY-003
TS-EMPTY-01	REQ-14	COND-EMPTY-01	Không checkout cart rỗng bằng COD	Negative	P1	TC-EMPTY-001
TS-EMPTY-02	REQ-14	COND-EMPTY-01	Không checkout cart rỗng bằng ONLINE	Negative	P1	TC-EMPTY-002
TS-EMPTY-03	REQ-14	COND-EMPTY-02	Cart có item checkout được	Positive	P1	TC-EMPTY-003
TS-METHOD-01	REQ-16	COND-METHOD-01	Từ chối payment method WALLET	Negative	P1	TC-METHOD-001
TS-METHOD-02	REQ-16	COND-METHOD-01	Từ chối payment method BANK	Negative	P1	TC-METHOD-002
TS-METHOD-03	REQ-16	COND-METHOD-02	COD nằm trong danh sách method hợp lệ	Positive	P1	TC-METHOD-003

4.3. Traceability coverage

REQ	COND IDs	TS IDs	TC IDs	Run coverage
REQ-01	COND-CAT-01, COND-CAT-02	TS-CAT-01, TS-CAT-02, TS-CAT-03	TC-CAT-001, TC-CAT-002, TC-CAT-003	3/3
REQ-02	COND-CART-01, COND-CART-02	TS-CART-01, TS-CART-02, TS-CART-03	TC-CART-001, TC-CART-002, TC-CART-003	3/3
REQ-03	COND-PRICE-01, COND-PRICE-02	TS-PRICE-01, TS-PRICE-02, TS-PRICE-03	TC-PRICE-001, TC-PRICE-002, TC-PRICE-003	3/3
REQ-04	COND-AUTH-01, COND-AUTH-02	TS-AUTH-01, TS-AUTH-02, TS-AUTH-03	TC-AUTH-001, TC-AUTH-002, TC-AUTH-003	3/3
REQ-09	COND-SHP-01, COND-SHP-02	TS-SHP-01, TS-SHP-02, TS-SHP-03	TC-SHP-001, TC-SHP-002, TC-SHP-003	3/3
REQ-10	COND-PAY-01, COND-PAY-02	TS-PAY-01, TS-PAY-02, TS-PAY-03	TC-PAY-001, TC-PAY-002, TC-PAY-003	3/3
REQ-14	COND-EMPTY-01, COND-EMPTY-02	TS-EMPTY-01, TS-EMPTY-02, TS-EMPTY-03	TC-EMPTY-001, TC-EMPTY-002, TC-EMPTY-003	3/3
REQ-16	COND-METHOD-01, COND-METHOD-02	TS-METHOD-01, TS-METHOD-02, TS-METHOD-03	TC-METHOD-001, TC-METHOD-002, TC-METHOD-003	3/3

Tổng thiết kế: 8 requirements, 16 conditions, 24 scenarios/cases. Trong đó 10 case negative/rejection và 2 case boundary về ngưỡng shipping. Mọi case có expected riêng và được chạy độc lập.

5. DETAILED TEST CASES - NHÓM 1

Bảng ghi lại test data, steps, expected oracle, actual result và evidence reference. Các case được thực thi từ CSV; expected gốc không bị ghi đè.

REQ-01 - Catalog

Test Case / Scenario	Precondition & Test Data	Steps	Expected	Actual / Status / Evidence
TC-CAT-001 TS-CAT-01	Catalog mặc định có SKU A active fixture=catalog_default {"operation": "catalog_get", "sku": "A"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"sku": "A", "active": true}	{"active": true, "price": "200000", "sku": "A"} PASS / 0.082 ms EV-TC-CAT-001
TC-CAT-002 TS-CAT-02	SKU C được chuyển inactive trước thao tác fixture=catalog_default {"operation": "catalog_get", "sku": "C", "fixture_config": {"product_updates": {"C": {"active": false}}}}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "PRODUCT_UNAVAILABLE"}	Exception: ValueError / PRODUCT_UNAVAILABLE PASS / 0.035 ms EV-TC-CAT-002
TC-CAT-003 TS-CAT-03	Catalog không có SKU Z fixture=catalog_default {"operation": "catalog_get", "sku": "Z"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "PRODUCT_UNAVAILABLE"}	Exception: ValueError / PRODUCT_UNAVAILABLE PASS / 0.017 ms EV-TC-CAT-003

REQ-02 - Cart

Test Case / Scenario	Precondition & Test Data	Steps	Expected	Actual / Status / Evidence
TC-CART-001 TS-CART-01	Giỏ hàng ban đầu rỗng fixture=empty_cart {"operation": "cart_add", "sku": "A", "quantity": 2}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"lines": [{"sku": "A", "quantity": 2}], "subtotal": "400000"}	{"lines": [{"quantity": 2, "sku": "A"}], "subtotal": "400000", "subtotal_type": "Decimal"} PASS / 0.034 ms EV-TC-CART-001
TC-CART-002 TS-CART-02	Giỏ ban đầu có A số lượng 2 fixture=a2 {"operation": "cart_add", "sku": "A", "quantity": 3}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"lines": [{"sku": "A", "quantity": 5}], "subtotal": "1000000"}	{"lines": [{"quantity": 5, "sku": "A"}], "subtotal": "1000000", "subtotal_type": "Decimal"} PASS / 0.024 ms EV-TC-CART-002
TC-CART-003 TS-CART-03	Giỏ rỗng và quantity không hợp lệ fixture=empty_cart {"operation": "cart_add", "sku": "A", "quantity": 0}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "INVALID_QUANTITY"}	Exception: ValueError / INVALID_QUANTITY PASS / 0.05 ms EV-TC-CART-003

5. DETAILED TEST CASES - NHÓM 2

Bảng ghi lại test data, steps, expected oracle, actual result và evidence reference. Các case được thực thi từ CSV; expected gốc không bị ghi đè.

REQ-03 - Pricing

Test Case / Scenario	Precondition & Test Data	Steps	Expected	Actual / Status / Evidence
TC-PRICE-001 TS-PRICE-01	Giỏ có A x2 và B x1 fixture=a2_b1 {"operation": "cart_subtotal"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"subtotal": "700000", "subtotal_type": "Decimal"}	{"subtotal": "700000", "subtotal_type": "Decimal"} PASS / 0.021 ms EV-TC-PRICE-001
TC-PRICE-002 TS-PRICE-02	Giá A=0.1 và B=0.2 được cấu hình bằng Decimal fixture=a1_b1 {"operation": "cart_subtotal", "fixture_config": {"product_updates": {"A": {"price": "0.1"}, "B": {"price": "0.2"}}}}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"subtotal": "0.3", "subtotal_type": "Decimal"}	{"subtotal": "0.3", "subtotal_type": "Decimal"} PASS / 0.021 ms EV-TC-PRICE-002
TC-PRICE-003 TS-PRICE-03	Giá SKU A được cập nhật thành 199999.5 trước khi tính fixture=a2 {"operation": "cart_subtotal", "fixture_config": {"product_updates": {"A": {"price": "199999.5"}}}}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"subtotal": "399999.0", "subtotal_type": "Decimal"}	{"subtotal": "399999.0", "subtotal_type": "Decimal"} PASS / 0.016 ms EV-TC-PRICE-003

REQ-04 - Authentication

Test Case / Scenario	Precondition & Test Data	Steps	Expected	Actual / Status / Evidence
TC-AUTH-001 TS-AUTH-01	Customer chưa đăng nhập; giỏ có A x1 fixture=a1 {"operation": "checkout", "method": "COD", "fixture_config": {"authenticated": false}}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "LOGIN_REQUIRED", "assert_state": {"order_count": 0, "charge_count": 0}}	Exception: ValueError / LOGIN_REQUIRED PASS / 0.019 ms EV-TC-AUTH-001
TC-AUTH-002 TS-AUTH-02	Customer chưa đăng nhập; giỏ có A x1 fixture=a1 {"operation": "checkout", "method": "ONLINE", "approved": true, "fixture_config": {"authenticated": false}}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "LOGIN_REQUIRED", "assert_state": {"order_count": 0, "charge_count": 0}}	Exception: ValueError / LOGIN_REQUIRED PASS / 0.017 ms EV-TC-AUTH-002
TC-AUTH-003 TS-AUTH-03	Customer đã đăng nhập; giỏ hợp lệ fixture=a1 {"operation": "checkout", "method": "COD"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"status": "PENDING_COD", "order_count": 1, "customer_id": "U_TEST"}	{"charge_count": 0, "customer_id": "U_TEST", "discount": "0", "lines": [{"quantity": 1, "sku": "A"}], "order_count": 1, "order_id": "ORD-0001", "payment_method": "COD", "shipping": "30000", "status": "PENDING_COD", "total": "230000"} State: stock changed, order count changed PASS / 0.042 ms EV-TC-AUTH-003

5. DETAILED TEST CASES - NHÓM 3

Bảng ghi lại test data, steps, expected oracle, actual result và evidence reference. Các case được thực thi từ CSV; expected gốc không bị ghi đè.

REQ-09 - Shipping

Test Case / Scenario	Precondition & Test Data	Steps	Expected	Actual / Status / Evidence
TC-SHP-001 TS-SHP-01	Subtotal nội thành là 499999 fixture=a1_b1 {"operation": "shipping_calculate", "subtotal": "499999", "area": "inner"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"shipping_fee": "30000"}	{"fee_type": "Decimal", "shipping_fee": "30000"} PASS / 0.017 ms EV-TC-SHP-001
TC-SHP-002 TS-SHP-02	Subtotal nội thành là 500000 fixture=a1_b1 {"operation": "shipping_calculate", "subtotal": "500000", "area": "inner"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"shipping_fee": "0"}	{"fee_type": "Decimal", "shipping_fee": "0"} PASS / 0.015 ms EV-TC-SHP-002
TC-SHP-003 TS-SHP-03	Subtotal ngoại thành là 500000 fixture=a1_b1 {"operation": "shipping_calculate", "subtotal": "500000", "area": "outer"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"shipping_fee": "50000"}	{"fee_type": "Decimal", "shipping_fee": "50000"} PASS / 0.017 ms EV-TC-SHP-003

REQ-10 - Payment

Test Case / Scenario	Precondition & Test Data	Steps	Expected	Actual / Status / Evidence
TC-PAY-001 TS-PAY-01	Customer đăng nhập và cart hợp lệ fixture=a2 {"operation": "checkout", "method": "COD"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"status": "PENDING_COD", "order_count": 1, "charge_count": 0}	{"charge_count": 0, "customer_id": "U_TEST", "discount": "0", "lines": [{"quantity": 2, "sku": "A"}], "order_count": 1, "order_id": "ORD-0001", "payment_method": "COD", "shipping": "30000", "status": "PENDING_COD", "total": "430000"} State: stock changed, order count changed PASS / 0.025 ms EV-TC-PAY-001
TC-PAY-002 TS-PAY-02	Gateway trả approved true fixture=a2 {"operation": "checkout", "method": "ONLINE", "approved": true}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"status": "PAID", "order_count": 1, "charge_count": 1}	{"charge_count": 1, "customer_id": "U_TEST", "discount": "0", "lines": [{"quantity": 2, "sku": "A"}], "order_count": 1, "order_id": "ORD-0001", "payment_method": "ONLINE", "shipping": "30000", "status": "PAID", "total": "430000"} State: stock changed, order count changed, payment charge count changed PASS / 0.022 ms EV-TC-PAY-002
TC-PAY-003 TS-PAY-03	Gateway trả approved false fixture=a2 {"operation": "checkout", "method": "ONLINE", "approved": false}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "PAYMENT_DECLINED", "assert_state": {"order_count": 0, "charge_count": 0}}	Exception: ValueError / PAYMENT_DECLINED State: stock changed PASS / 0.022 ms EV-TC-PAY-003

5. DETAILED TEST CASES - NHÓM 4

Bảng ghi lại test data, steps, expected oracle, actual result và evidence reference. Các case được thực thi từ CSV; expected gốc không bị ghi đè.

REQ-14 - Cart Integrity

Test Case / Scenario	Precondition & Test Data	Steps	Expected	Actual / Status / Evidence
TC-EMPTY-001 TS-EMPTY-01	Cart không có item fixture=empty_cart {"operation": "checkout", "method": "COD"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "EMPTY_CART", "assert_state": {"order_count": 0}}	Exception: ValueError / EMPTY_CART PASS / 0.012 ms EV-TC-EMPTY-001
TC-EMPTY-002 TS-EMPTY-02	Cart không có item fixture=empty_cart {"operation": "checkout", "method": "ONLINE", "approved": true}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "EMPTY_CART", "assert_state": {"order_count": 0}}	Exception: ValueError / EMPTY_CART PASS / 0.013 ms EV-TC-EMPTY-002
TC-EMPTY-003 TS-EMPTY-03	Cart có A x1 fixture=a1 {"operation": "checkout", "method": "COD"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"status": "PENDING_COD", "order_count": 1}	{"charge_count": 0, "customer_id": "U_TEST", "discount": "0", "lines": [{"quantity": 1, "sku": "A"}], "order_count": 1, "order_id": "ORD-0001", "payment_method": "COD", "shipping": "30000", "status": "PENDING_COD", "total": "230000"} State: stock changed, order count changed PASS / 0.02 ms EV-TC-EMPTY-003

REQ-16 - Payment Safety

Test Case / Scenario	Precondition & Test Data	Steps	Expected	Actual / Status / Evidence
TC-METHOD-001 TS-METHOD-01	Method ngoài COD và ONLINE fixture=a1 {"operation": "checkout", "method": "WALLET"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "INVALID_PAYMENT_METHOD", "assert_state": {"order_count": 0, "charge_count": 0}}	Exception: ValueError / INVALID_PAYMENT_METHOD State: stock changed PASS / 0.018 ms EV-TC-METHOD-001
TC-METHOD-002 TS-METHOD-02	Method ngoài COD và ONLINE fixture=a1 {"operation": "checkout", "method": "BANK"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"exception": "INVALID_PAYMENT_METHOD", "assert_state": {"order_count": 0, "charge_count": 0}}	Exception: ValueError / INVALID_PAYMENT_METHOD State: stock changed PASS / 0.017 ms EV-TC-METHOD-002
TC-METHOD-003 TS-METHOD-03	COD là method hỗ trợ fixture=a1 {"operation": "checkout", "method": "COD"}	1. fixture() mới. 2. Gọi operation theo input_json. 3. So sánh return/exception với expected. 4. Chụp snapshot state.	{"status": "PENDING_COD", "order_count": 1}	{"charge_count": 0, "customer_id": "U_TEST", "discount": "0", "lines": [{"quantity": 1, "sku": "A"}], "order_count": 1, "order_id": "ORD-0001", "payment_method": "COD", "shipping": "30000", "status": "PENDING_COD", "total": "230000"} State: stock changed, order count changed PASS / 0.018 ms EV-TC-METHOD-003

6. PYTHON TEST ARCHITECTURE

Bộ test gồm hai module Python và một file CSV. ecommerce_app.py giữ module nghiệp vụ theo Instruction; test_runner.py đọc test data, tạo fixture độc lập, dispatch function, so sánh expected/actual và xuất report. Không cần thư viện ngoài.

6.1. Fixture / Test Data Setup

```
def fixture(fixture_key="a1", config=None):
    service = Checkout()
    customer = Customer("U_TEST", authenticated=...)
    cart = Cart(service.catalog)
    # apply product_updates, then add fixture lines
    return service, customer, cart
```

Mỗi lần gọi fixture() tạo Checkout/Catalog, Customer và Cart mới. Các fixture key trong bộ test: catalog_default (không có line), empty_cart, a1 (A×1), a2 (A×2), a2_b1 (A×2+B×1), a1_b1 (A×1+B×1). Per-row fixture_config có thể thay active hoặc giá Decimal trước khi thao tác. Điều này tránh stock, price, used_codes hoặc order từ case trước ảnh hưởng case sau.

6.2. CSV Reader và schema validation

```
with path.open("r", encoding="utf-8-sig", newline="") as file:
    reader = csv.DictReader(file)
    missing = REQUIRED_COLUMNS - set(reader.fieldnames)
    if missing: raise ValueError(...)
    rows = list(reader)
# reject empty/duplicate case IDs; retain malformed JSON rows as BLOCKED
```

CSV chứa đủ các cột theo Requirement: test_case_id, requirement_id, condition_id, scenario_id, title, priority, precondition, fixture_key, input_json, expected_type, expected_json, execution_status, actual_result, error_message và evidence_reference. Runner dùng utf-8-sig để đọc CSV Excel-friendly.

6.3. Test Dispatcher

```
if operation == "catalog_get":
    product = app.catalog.get(inputs["sku"])
elif operation == "cart_add":
    cart.add(inputs["sku"], inputs["quantity"])
elif operation == "cart_subtotal":
    subtotal = cart.subtotal()
elif operation == "shipping_calculate":
    fee = app.shipping.calculate(D(inputs["subtotal"]), inputs["area"])
elif operation == "checkout":
    order = app.place(customer, cart, method=..., approved=..., key=...)
```

Dispatcher chọn function đúng module: Catalog.get, Cart.add/subtotal, Shipping.calculate và Checkout.place. Input được parse từ JSON trong từng dòng CSV; không hard-code expected theo actual implementation.

6.4. Assertion, exception handling và state

```
if expected_type == "exception":
    passed = exception_type == "ValueError" and error_message == expected["exception"]
elif expected_type == "result":
    passed = not exception_type and subset_matches(actual, expected)
# setup/parse failure -> BLOCKED; mismatch -> FAIL; match -> PASS
```

Trước/sau mỗi operation, runner chụp stock, order_count, charge_count và used_codes. Với expected exception, runner phân biệt lỗi nghiệp vụ dự kiến với lỗi ngoài mong đợi. Input JSON hỏng/setup không tạo được precondition được đánh dấu BLOCKED, không tự bỏ khỏi tổng số case.

6.5. Reporter

• reports/test_execution.csv: actual, exception, state snapshot, status, execution time, timestamp và evidence reference.

• reports/summary_by_requirement.csv: số PASS/FAIL/BLOCKED theo REQ.

• reports/run_summary.json: total, status count, pass rate, selected requirements và thời điểm chạy.

• reports/test_dashboard.html: KPI, status bar, bảng theo requirement và chi tiết từng test case.

7. EXECUTION EVIDENCE & TEST RUN INFORMATION

Bộ test được chạy bằng lệnh `python test_runner.py` trong thư mục repository, ngày 2026-11-02T21:39:28+07:00. Đây là log thực thi local từ chính source/CSV đính kèm; không phải ảnh Colab. Notebook STLab02_EduCommerce.ipynb đi kèm để chạy lại trên Google Colab sau khi upload project.

Evidence ID	Artifact / nội dung	Dấu vết được cung cấp
E01	Cấu trúc source và notebook	ecommerce_app.py, test_runner.py, test_cases.csv, README.md, notebook và reports/ trong ZIP.
E02	Selected REQs / traceability	Bảng REQ → COND → TS → TC tại Mục 4; traceability_matrix.csv trong repository.
E03	Fixture và state	Code fixture() ở ecommerce_app.py; self_test_runner.py xác nhận hai fixture độc lập.
E04	CSV reader / schema	test_runner.py: load_cases(), _load_json(); đọc utf-8-sig, required columns, duplicate ID.
E05	Dispatcher	test_runner.py: execute_case() nhánh catalog_get/cart_add/cart_subtotal/shipping_calculate/checkout.
E06	Exception / assertion	test_runner.py: exception oracle, subset match, PASS/FAIL/BLOCKED.
E07	Execution output	reports/console_output.txt và 24 rows reports/test_execution.csv.
E08	Report / chart	reports/test_dashboard.html, summary_by_requirement.csv và biểu đồ trạng thái tại Mục 8.
E09	State observation	TC-PAY-003: before/after stock, PAYMENT_DECLINED, không tạo order. Phân tích ở Mục 9.
E10	GitHub	Chưa có ảnh public repository vì chưa kết nối tài khoản GitHub; xem GITHUB_UPLOAD_GUIDE.md. Không tạo ảnh giả.

7.1. Console output - run thật từ repository

```
Executed 24 | PASS 24 | FAIL 0 | BLOCKED 0
Pass rate: 100.0%
Execution CSV: /mnt/data/STLab02_EduCommerce/reports/test_execution.csv
Dashboard: /mnt/data/STLab02_EduCommerce/reports/test_dashboard.html
TC-CAT-001 | REQ-01 | PASS | 0.082 ms |
TC-CAT-002 | REQ-01 | PASS | 0.035 ms | PRODUCT_UNAVAILABLE
TC-CAT-003 | REQ-01 | PASS | 0.017 ms | PRODUCT_UNAVAILABLE
TC-CART-001 | REQ-02 | PASS | 0.034 ms |
...
TC-EMPTY-002 | REQ-14 | PASS | 0.013 ms | EMPTY_CART
TC-EMPTY-003 | REQ-14 | PASS | 0.02 ms |
TC-METHOD-001 | REQ-16 | PASS | 0.018 ms | INVALID_PAYMENT_METHOD
TC-METHOD-002 | REQ-16 | PASS | 0.017 ms | INVALID_PAYMENT_METHOD
TC-METHOD-003 | REQ-16 | PASS | 0.018 ms |
```

Timestamp từng case được ghi theo múi giờ Asia/Ho_Chi_Minh trong reports/test_execution.csv. File log đầy đủ nằm trong repository. Lần chạy này không có BLOCKED vì toàn bộ CSV hợp lệ.

8. TEST REPORT: SUMMARY, CHART & COVERAGE

Metric	Count	Interpretation
Total cases	24	24 dòng CSV được đọc và chạy.
PASS	24	Actual khớp expected.
FAIL	0	Không có mismatch trong 8 REQ được chọn.
BLOCKED	0	Không có case bị chặn bởi setup/data.
Pass rate	100.0%	24 PASS / 24 executed.
Conditions / scenarios	16 / 24	2 conditions mỗi REQ; 3 scenarios mỗi REQ.
Negative / boundary	10 / 2	Negative/rejection scenarios và boundary cho shipping threshold.

8.1. Status distribution

Executed test cases



PASS 24 FAIL 0 BLOCKED 0

8.2. Summary by requirement

Requirement	PASS	FAIL	BLOCKED	Total	Pass rate
REQ-01	3	0	0	3	100%
REQ-02	3	0	0	3	100%
REQ-03	3	0	0	3	100%
REQ-04	3	0	0	3	100%
REQ-09	3	0	0	3	100%
REQ-10	3	0	0	3	100%
REQ-14	3	0	0	3	100%
REQ-16	3	0	0	3	100%

8.3. Diễn giải kết quả

Tất cả 24 test case trong phạm vi được chọn đều đạt expected. Điều này xác nhận các behavior đã kiểm tra của bản ứng dụng mô phỏng phù hợp với acceptance criteria tương ứng. Kết quả không chứng minh những requirement không nằm trong scope đã đúng; cũng không chứng minh các tình huống concurrency, database transaction hay real payment gateway.

9. ROOT CAUSE / DEFECT CANDIDATE / RETEST

Không có test case FAIL trong 8 REQ đã chọn. Theo yêu cầu, phần này ghi evidence và các rủi ro còn lại. Ngoài ra, state snapshot cho thấy một observation liên module cần xử lý, nhưng không tính lại pass rate của REQ-10/REQ-16.

9.1. Quan sát từ TC-PAY-003 và TC-METHOD-001

Test case	Expected theo REQ đã chọn	Observed state trước → sau	Status
TC-PAY-003 / REQ-10	PAYMENT_DECLINED; không tạo order/charge.	Stock A: 20 → 18; order_count 0 → 0; charge_count 0 → 0.	PASS REQ-10; ghi nhận stock side effect.
TC-METHOD-001 / REQ-16	INVALID_PAYMENT_METHOD; không tạo order/charge.	Stock A: 20 → 19; order_count 0 → 0; charge_count 0 → 0.	PASS REQ-16; ghi nhận stock side effect.

9.2. Root-cause analysis - observation ngoài phạm vi chấm chính

Đường thực thi: ecommerce_app.py → Checkout.place() (khoảng dòng 189-193) gọi Inventory.reserve(cart.lines) trước, sau đó Payments.collect(...). Khi collect() ném PAYMENT_DECLINED hoặc INVALID_PAYMENT_METHOD, hàm thoát bằng exception. Không có try/except/finally gọi Inventory.release(cart.lines), nên stock đã reserve không được trả lại. Đây là nguyên nhân trực tiếp khớp với snapshot thực tế.

```
ecommerce_app.py - Checkout.place()
self.inventory.reserve(cart.lines)
...
payment_state = self.payments.collect(total, method, approved, key)

# Payments.collect() may raise ValueError; no release() is reached.
```

Phạm vi phân loại: REQ-08 là requirement inventory rollback đã được Instruction làm mẫu và bị yêu cầu không chọn lại. Vì vậy observation này không được dùng để tạo thêm selected REQ hay tính thành FAIL trong pass rate của report. Tuy nhiên, nên báo người hướng dẫn/BA và mở test regression riêng khi được phép mở rộng scope.

9.3. Suggested fix & regression candidate

• Validate payment method trước khi reserve stock; không reserve nếu method không thuộc COD/ONLINE.

• Với payment declined hoặc lỗi sau reserve, dùng transaction/try-except có rollback rõ ràng; gọi Inventory.release(cart.lines) đúng một lần khi chưa commit order.

• Test lại: declined → stock không đổi, không order, không charge; unsupported method → stock không đổi, không order; approved → stock giảm đúng một lần và order PAID.

• Không sửa source trong bài thực thi này vì mục tiêu là kiểm thử ứng dụng mẫu và giữ nguyên actual evidence.

9.4. Hai rủi ro chưa chạy do nằm ngoài scope được chọn

Risk	Vị trí nghi vấn trong source	Vì sao chưa kết luận trong bộ test này
Voucher threshold tại đúng 500.000 (REQ-06)	Promotions.discount(): `if subtotal <= voucher.min_subtotal` (dòng khoảng 131).	REQ-06 là worked example bị loại khỏi selected scope; cần chạy test oracle boundary riêng nếu được giao.
Idempotency retry (REQ-11)	Checkout.place() kiểm tra key sau Inventory.reserve() (dòng khoảng 189-192).	REQ-11 là worked example bị loại; retry có thể làm phát sinh stock side effect dù order không nhân đôi.

10. PROBLEM SOLVING (PHẦN 3) - TÌNH HUỐNG 3.1–3.4

10.1. Hai SKU, một SKU thiếu tồn kho

Với cart A=2, B=100 trong khi A đủ stock nhưng B không đủ, expected là OUT_OF_STOCK và không SKU nào bị trừ. Inventory.reserve() hiện kiểm tra đủ hàng cho tất cả line trước khi bắt đầu decrement; cấu trúc hai vòng lặp này bảo vệ tính atomic trong mô hình đơn luồng. Risk còn lại là concurrency vì lab không mô phỏng hai checkout đồng thời.

Nội dung	Phân tích / đề xuất
File/function	ecommerce_app.py → Inventory.reserve(lines); được gọi từ Checkout.place().
TC bổ sung 1	A=2, B=100; B stock thấp hơn 100. Expected OUT_OF_STOCK; stock A/B trước-sau không đổi.
TC bổ sung 2	A/B đều đủ stock. Expected reserve thành công; stock từng SKU giảm đúng quantity yêu cầu.
Evidence	Stock snapshot trước/sau; exception code; CartLine và số lượng từng SKU.

10.2. Voucher chỉ đánh dấu used sau thành công

Trong source, customer.used_codes.add(voucher) nằm sau khi Order được tạo và lưu. Vì vậy nếu ONLINE declined làm hàm raise trước đoạn này, voucher không bị đánh dấu used. Đây là điểm đúng theo acceptance rule, nhưng state stock vẫn có vấn đề riêng như Mục 9.

Nội dung	Phân tích / đề xuất
File/function	ecommerce_app.py → Checkout.place(), đoạn tạo order và cập nhật used_codes (dòng khoảng 205-208).
TC bổ sung 1	SAVE10 + ONLINE approved=False. Expected PAYMENT_DECLINED; không order; SAVE10 không nằm trong used_codes.
TC bổ sung 2	SAVE10 + COD success. Expected một order; SAVE10 được thêm vào used_codes sau thành công.
Evidence	used_codes trước/sau, order_count, payment status, exception.

10.3. Mã thanh toán lạ WALLET

Payments.collect() từ chối method khác COD/ONLINE. Tuy nhiên Checkout.place() gọi reserve trước collect, nên invalid method xuất hiện sau khi stock đã bị trừ. Nên validate method sớm hoặc đảm bảo rollback khi collect ném exception.

TC gợi ý	Input	Expected / Evidence
PS-3A	method=WALLET; cart A×1.	INVALID_PAYMENT_METHOD; không order/charge; theo transactional safety, stock cần giữ nguyên. Snapshot stock before/after.
PS-3B	method=COD; cart A×1.	Order PENDING_COD; stock giảm đúng một; không tạo charge online.

10.4. Giá sản phẩm thay đổi sau add-to-cart

Requirement hiện tại không nói giá được khóa ở thời điểm add-to-cart hay lấy giá mới nhất khi checkout. Code Cart.subtotal() truy vấn giá hiện tại từ Catalog, vì thế thay đổi price trước khi gọi subtotal sẽ thay đổi giá trị cart. Chưa thể kết luận đó là expected đúng nếu chưa có câu trả lời BA/PO.

• Question: price lock tại thời điểm nào? Có cần thông báo khi giá thay đổi không? Có cần user xác nhận total lần cuối trước payment không?

• TC-3.4A: add A khi giá 200.000; cập nhật giá 220.000 trước checkout; expected để pending cho đến khi BA xác nhận.

• TC-3.4B: thay đổi giá sau khi trang checkout hiển thị total nhưng trước Place Order; xác nhận total/payment request không bị lệch.

• Evidence: lịch sử price, timestamp add/cart/checkout, total UI, order total và payment amount.

10. PROBLEM SOLVING (PHẦN 3) - TÌNH HUỐNG 3.5–3.8

10.5. Cùng SKU nhập hai lần

Add A×2 rồi A×3 phải tạo một CartLine duy nhất cho SKU A, quantity 5; subtotal là 5 × 200.000 = 1.000.000 VND. Cart.add() hiện tìm line cùng SKU và cộng dồn quantity.

Test	Steps / data	Expected
PS-5A	A×2 rồi A×3	Một line A quantity 5; subtotal 1.000.000.
PS-5B	A×2, B×1, A×3	Hai line; A quantity 5, B quantity 1; subtotal 1.300.000.
PS-5C	Add A quantity 0 hoặc -1	INVALID_QUANTITY; cart giữ nguyên.

10.6. Data-driven tests bị phụ thuộc nhau

Nếu dùng chung một Checkout/Catalog, thay đổi stock, price, used_codes, orders hoặc charges ở case trước sẽ ảnh hưởng case sau. Runner khắc phục bằng cách gọi fixture() trong mỗi execute_case(), không dùng mutable global state.

• TC-6A: gọi fixture() hai lần; kiểm tra hai Checkout/Cart khác identity và stock/price mặc định độc lập.

• TC-6B: thay stock/price/used_codes trên fixture A; fixture B vẫn giữ giá trị ban đầu.

• Chạy suite theo thứ tự đảo ngược và so sánh status vector; kết quả không nên phụ thuộc thứ tự.

10.7. Dashboard PASS dù CSV bị lỗi parse

Nếu 20 dòng CSV có 3 dòng malformed mà runner bỏ qua, 17 PASS không thể được báo là pass rate 100% của 20 dòng đã cung cấp. Denominator phải là số dòng input được yêu cầu xử lý. Runner hiện giữ row parse JSON lỗi dưới trạng thái BLOCKED, và từ chối duplicate ID/missing schema; không im lặng bỏ dòng.

Test kiểm completeness	Expected
PS-7A: một row input_json hỏng	Row vẫn xuất hiện với BLOCKED; total vẫn giữ nguyên; pass rate tính trên toàn bộ row.
PS-7B: duplicate test_case_id	load_cases() ném DUPLICATE_TEST_IDS; không xuất report partial gây hiểu lầm.
Evidence	Input row count, execution row count, status totals, error message; self_test_runner.py có kiểm tra tương ứng.

10.8. Ngưỡng miễn phí shipping đổi từ 500k lên 700k

Change impact: cập nhật acceptance rule REQ-09; COND-SHP-01; scenario và các case boundary; data set; Shipping.calculate(); checkout total và regression scope. Rule outer 50.000 không đổi. Instruction quy định shipping dùng subtotal trước discount.

TC regression	Input	Expected
PS-8A	Inner subtotal 699.999	Shipping 30.000.
PS-8B	Inner subtotal 700.000	Shipping 0.
PS-8C	Outer subtotal 700.000	Shipping 50.000.

11. INTERVIEW QUESTIONS (PHẦN 4) - CÂU 1–10

1. Requirement, Condition, Scenario, Case, Execution

Requirement là rule; Condition là khía cạnh cần kiểm; Scenario là tình huống nghiệp vụ; Test Case gồm dữ liệu/bước/expected; Execution ghi actual, evidence và status. Ví dụ REQ-09 → condition inner threshold → scenario subtotal đúng 500.000 → case nhập 500.000, area inner, expected fee 0 → chạy và ghi actual.

2. Test oracle là gì? Vì sao không lấy actual làm expected?

Test oracle là căn cứ xác định kết quả đúng. Expected phải lấy từ acceptance criteria hoặc rule đã xác nhận. Lấy actual của code làm expected sẽ biến lỗi implementation thành "đúng" và làm mất tính độc lập của kiểm thử.

3. Khi nào dùng equivalence partitioning và boundary value analysis?

Dùng khi input có nhóm tương đương hoặc điểm ranh giới. Với Shipping, inner subtotal dưới 500.000 và từ 500.000 trở lên là hai nhóm; kiểm thêm 499.999, 500.000, 500.001 giúp bắt sai dấu so sánh. Đây là kỹ thuật mở rộng kiến thức, không phải phần làm sâu trong selected REQ của bài này.

4. Positive, Negative và Edge case của Cart.add()

Positive: add A quantity 2. Negative: quantity 0 hoặc -1 bị từ chối. Edge: add cùng SKU nhiều lần sát giới hạn quantity nếu hệ thống có quy định giới hạn; hiện requirement chưa nêu max quantity nên không tự tạo expected cho ngưỡng đó.

5. Fixture là gì? Vì sao mỗi case cần riêng?

Fixture là hàm dựng precondition/dữ liệu ban đầu. Mỗi case dùng fixture mới giúp stock, giá, used_codes, order và payment charge không bị case trước làm thay đổi.

6. Test isolation khác reset data sau test thế nào?

Isolation bảo đảm case không chia sẻ trạng thái mutable và có thể chạy độc lập, kể cả chạy song song/đổi thứ tự. Reset chỉ khôi phục dữ liệu sau khi case chạy; nếu reset thiếu state hoặc lỗi giữa chừng thì test vẫn có thể nhiễm chéo.

7. Phân biệt expected ValueError với exception ngoài mong đợi

Runner phải so sánh cả exception type và message với expected. Nếu case dự kiến INVALID_QUANTITY và nhận đúng ValueError với message INVALID_QUANTITY thì PASS; nếu nhận KeyError hoặc message khác thì FAIL. Exception từ setup/test data nên được phân biệt thành BLOCKED.

8. Vì sao kiểm return value chưa đủ với Inventory/Payments/Orders?

Vì function có thể trả đúng trạng thái nhưng side effect sai: stock trừ nhiều lần, charge duplicate, order tạo trước khi payment approved, hoặc voucher bị đánh dấu used sớm. Cần snapshot state trước/sau.

9. Vì sao Decimal thay float khi tính tiền?

Decimal biểu diễn thập phân theo cơ số 10, phù hợp tiền tệ; float dùng nhị phân nên một số số thập phân không biểu diễn chính xác. Điều này có thể tạo chênh lệch khi cộng/nhân nhiều lần. Rounding cũng phải có rule rõ ràng.

10. Data-driven testing là gì?

Test logic được tách khỏi test data; mỗi dòng CSV chứa ID, REQ/COND/TS, input, expected. Runner đọc dòng, dựng fixture, dispatch function, so sánh kết quả và ghi status; cùng một code có thể chạy nhiều bộ dữ liệu.

11. INTERVIEW QUESTIONS (PHẦN 4) - CÂU 11–20

11. Traceability Matrix phục vụ gì?

Traceability cho biết requirement nào được kiểm bởi condition/scenario/case nào và đã chạy chưa. Khi requirement thay đổi, tester xác định nhanh test bị ảnh hưởng, tránh bỏ coverage hoặc chạy test không liên quan.

12. Priority và Severity khác nhau?

Priority là mức cần ưu tiên chạy/sửa theo thời điểm và mục tiêu release; Severity là mức độ tác động của defect. P0 test có thể liên quan một business critical path; severity của defect phụ thuộc tác động thực tế.

13. Unit vs Integration vs System; Checkout.place() thuộc mức nào?

Unit test kiểm một function/class độc lập; Integration test kiểm tương tác giữa Checkout, Inventory, Payments, Catalog; System test kiểm luồng end-to-end. Test gọi Checkout.place() trong in-memory module thường ở mức service/integration hơn là test giao diện/system hoàn chỉnh.

14. State transition COD/ONLINE và PAID

COD tạo order với trạng thái PENDING_COD. ONLINE chỉ được PAID khi approved=True. Khi approved=False, payment bị từ chối và không tạo order theo REQ-10. Không suy ra trạng thái PAID chỉ vì request gửi tới gateway.

15. Idempotency là gì? Vì sao retry kiểm side effect?

Cùng idempotency key đại diện cùng một yêu cầu; retry không tạo order/charge hoặc trừ stock thêm lần nữa. Chỉ kiểm order_id là chưa đủ vì stock hoặc payment charge vẫn có thể bị lặp.

16. FAIL do product hay test script/fixture?

Đầu tiên xác minh test data và precondition; chạy lại trên fixture sạch; đối chiếu expected với requirement; xem actual/exception và state. Nếu setup sai thì sửa test và đánh dấu run không hợp lệ; nếu setup đúng và app sai rule thì mới tạo defect.

17. Gradio dashboard thay raw evidence không?

Không. Dashboard giúp xem nhanh phân bố kết quả và lọc; raw CSV, log, timestamp, input, expected/actual và state snapshot vẫn cần để tái hiện và review.

18. Sau sửa bug chạy Sanity và Regression thế nào?

Sanity tập trung vào function/flow vừa sửa để xác nhận fix hoạt động. Sau đó chọn focused regression theo dependency và risk, gồm các flow có dùng cùng module/state; không chạy toàn bộ suite nếu không cần nhưng phải ghi rõ phần chưa chạy.

19. Repository cần gì để tái hiện kết quả Colab?

Source application, runner, CSV expected gốc, README với Python/package/version và lệnh chạy, notebook nếu dùng Colab, report execution mẫu, report PDF và không chứa token/secret. Ghi rõ cách tạo fixture và nơi lưu actual output.

20. Nếu chỉ có 5 case thì chọn theo risk?

Ưu tiên P0: payment approved/declined, authentication gate, shipping threshold, cart empty/stock availability tùy release change. Xem severity nghiệp vụ, khả năng xảy ra, dependency, lịch sử lỗi và change impact; ghi các risk chưa được cover.

11.1. CODE INTERVIEW - PHÂN TÍCH / SỬA LỖI

A. Exception oracle - expected ValueError nhưng runner đánh FAIL

Runner không được đánh FAIL chỉ vì exception xuất hiện. Cần khai báo expected_type=exception và expected_json chứa exception/message. Khi chạy, bắt exception, kiểm tra đúng class ValueError và đúng message INVALID_QUANTITY. Đúng cả hai mới PASS; exception khác hoặc không có exception là FAIL. Nếu parse/setup lỗi trước operation thì BLOCKED.

```
try:
    cart.add("A", 0)
except ValueError as exc:
    actual_error = str(exc)
    passed = actual_error == expected["exception"]  # INVALID_QUANTITY
else:
    passed = False  # expected exception did not occur
```

B. False positive - chỉ kiểm order_id khác None

Đây là false positive / assertion không đủ mạnh. Order ID tồn tại không chứng minh total, payment state, lines, stock hay charge đúng. Bổ sung assertion theo acceptance criteria và kiểm state:

```
assert order.order_id is not None
assert order.customer_id == customer.customer_id
assert order.total == subtotal - discount + shipping
assert order.status == expected_payment_status
assert order.lines == expected_lines
assert stock_after == expected_stock_after
assert len(app.orders) == 1
```

C. Shared fixture - một fixture dùng cho mọi case

Rủi ro: case trước trừ stock, đổi giá, dùng voucher hoặc tạo order/charge làm case sau có expected sai. Cách kiểm: chạy fixture hai lần, kiểm object identity và state mặc định; thay đổi fixture A rồi xác nhận fixture B không bị ảnh hưởng. Sửa bằng cách tạo fixture trong execute_case(), không tạo global fixture ở đầu notebook.

```
def execute_case(row):
    app, customer, cart = fixture(row["fixture_key"], config)
    # run one test; do not reuse app/customer/cart in next row
```

12. SELF-REVIEW & GITHUB REPOSITORY

12.1. Self-review

• Đã chọn 8 requirement mới; không dùng REQ-06/REQ-08/REQ-11 làm selected scope.

• Có 16 Test Conditions, 24 Test Scenarios và 24 Detailed Test Cases (3 case/REQ).

• Có 10 negative/rejection cases và 2 boundary scenarios cho shipping threshold.

• Đã chạy 24 cases tự động bằng Python; tổng hợp 24 PASS, 0 FAIL, 0 BLOCKED.

• Có fixture isolation; CSV schema validation; duplicate-ID rejection; expected exception oracle; before/after state snapshot.

• Có execution CSV, summary theo REQ, JSON run summary và HTML dashboard.

• Đã phân tích stock side effect sau payment declined/invalid method; nêu rõ đây là observation ngoài selected scope.

• Đã trả lời đủ 8 problem-solving cases, 20 interview questions và 3 code-review cases.

• Chưa có URL GitHub public thực tế; cần tạo repository bằng tài khoản cá nhân và cập nhật URL vào PDF.

12.2. Remaining risks

• Chưa chạy trên website hoặc database thật; mô hình in-memory không kiểm tra concurrency/transaction database.

• Chưa có gateway thật, callback, timeout, network retry và refund/compensation.

• REQ-05, REQ-07, REQ-12, REQ-13, REQ-15 và ba requirement worked example không nằm trong selected scope của bộ 24 case.

• GitHub public link và ảnh chụp repository cần hoàn tất sau khi đăng nhập/publish.

12.3. Repository contents

File / folder	Vai trò
ecommerce_app.py	Ứng dụng EduCommerce in-memory theo Instruction.
test_runner.py	CSV reader, fixture dispatch, assertion, exception handling và reporter.
test_cases.csv	24 case, expected gốc và trace IDs.
traceability_matrix.csv	Mapping REQ → COND → TS → TC.
self_test_runner.py	3 quality checks cho malformed JSON, duplicate ID và fixture isolation.
STLab02_EduCommerce.ipynb	Notebook khởi chạy trên Google Colab.
README.md / GITHUB_UPLOAD_GUIDE.md	Hướng dẫn chạy và tạo repository GitHub cho người mới.
reports/	Actual execution CSV, summary, JSON, console log, HTML dashboard.
report/	PDF report cuối cùng đã được đóng gói trong report/STLab02_Report.pdf.

12.4. GitHub URL & publish steps

GitHub URL hiện trạng: CHƯA PUBLISH. Không có phiên kết nối GitHub được xác nhận, nên báo cáo không ghi URL giả. Repository dự kiến: stlab02-educommerce-testing (đây là tên repo đề xuất, chưa phải link truy cập).

• Tải ZIP project, giải nén; cài GitHub Desktop và đăng nhập GitHub.

• GitHub Desktop → File → New repository; đặt tên stlab02-educommerce-testing; chọn local path.

• Copy toàn bộ nội dung project vào thư mục repo; kiểm tra ecommerce_app.py nằm ngay ở root.

• Commit to main với message Initial commit - STLab02 EduCommerce testing.

• Publish repository; bỏ chọn Keep this code private để public (chỉ khi không có dữ liệu nhạy cảm).

• Mở View on GitHub, copy URL thật; thêm PDF vào thư mục report/, commit và Push origin.

• Cập nhật URL thật vào trang đầu PDF và email nộp bài.